MSBA 402 – Fall 2026

Intro to SQL and Basic Data Management

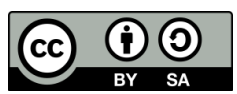
Lecture 4

October 26, 2026

Prof. Ryan Rosario

Anderson School of Management – Business Analytics

University of California, Los Angeles



This work is published under a Creative Commons BY-SA license.

Outline

- Querying Multiple Relations with Joins
  - Cardinality of Relationships
- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Joins: Querying Multiple Relations

Data is often spread throughout a database in many different places, and we need to be able to "tie together" data from these places into one result set.

This operation of combining two tables into one result set based on some common attribute(s) is called a **join**.

Joins: Querying Multiple Relations

We perform a **join** when we want to merge rows from two different tables R and S based on some **contextually/business related attribute(s)** in both tables.

The result, in the output, is a set of rows with data from both tables.

Joins: Querying Multiple Relations

The common attributes between the tables:

- are referred to as a **join key**
- do not need to enforce uniqueness
- *can* enforce uniqueness via `PRIMARY KEY` or `UNIQUE`
- *can* be used to enforce referential integrity but only if they are marked as `FOREIGN KEY`
- `FOREIGN KEY`s can be join keys but the purpose of an FK has nothing to do with joins.

The term **key** is overloaded in databases.

Joins: Querying Multiple Relations



An Introductory Example

We have two tables, one representing `student` and the other `score`. We want to produce output that shows the student info and the midterm scores together, for, say analysis.

`student`

id	name	level
1	Alice	UFR
2	Bob	USO
3	Charlie	UJR
4	Diana	USR
5	Eve	UFR

`score`

id	score
1	94
2	81
3	77
4	65
5	58



`student` ⋈ `score`

id	name	level	score
1	Alice	UFR	94
2	Bob	USO	81
3	Charlie	UJR	77
4	Diana	USR	65
5	Eve	UFR	58

Why Join at All?

Some questions:

- Why can't we just have a bunch of wider tables, one for each use case?[1]
- Isn't doing all of these joins slow?

We will actually come back to the second point a few times...

[1] This is called a **denormalized** or **exploded** table.

Separation of Concerns

Good database design relies on the concept of **separation of concerns** to:

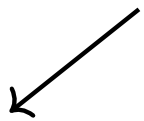
1. reduce redundancy
2. allow flexibility and a wider variety of use cases
3. prevent inconsistencies among these tables
4. prevent insert/update/delete anomalies

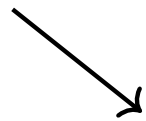
We enforce separation of concerns with **normalization** which creates a table for each component of business logic.

Separation of Concerns

`student` ⋈ `score`

id	name	level	score
1	Alice	UFR	94
2	Bob	USO	81
3	Charlie	UJR	77
4	Diana	USR	65
5	Eve	UFR	58





`student`

id	name	level
1	Alice	UFR
2	Bob	USO
3	Charlie	UJR
4	Diana	USR
5	Eve	UFR

`score`

id	score
1	94
2	81
3	77
4	65
5	58

Separation of Concerns

Good database design, via separation of concerns (a.k.a normalization or modularity):

- allows more flexibility for different use cases
- is more efficient
- enforces data integrity

We can then use the results in a spreadsheet, R, Pandas or even in the database.[2]

The join is the one operation that separates the relational model from others (e.g. document, graph).

[2] There are several ways to do this.

A Theoretical, but Helpful Device

For an **inner** join (the only kind we have seen so far), a join is simply a series of two steps on tables R and S:[3]

1. create all-pairs of rows from both tables (Cartesian product, $R \times S$)
2. filter (σ) the result from (1) on a particular condition ψ similar to `WHERE` ($\sigma_{\psi}(R \times S)$) but **as the join is being executed**.

That's it. Remember this strategy and the order of execution.

[3] A natural join takes no condition: it matches on every column with the same name in both tables, and keeps one copy of each. Outer joins, which we see next lecture, add back rows that have no match.

Example 1

Suppose we have the following two tables:

$R =$

A	B
α	β
γ	δ
ϵ	ζ

$S =$

A	C	D
α	η	θ
γ	ι	κ

Let's first compute the **Cartesian product** to get all pairs of rows.

Example 1

Next, use the common column names to perform the join...

It does not matter **where** in each table the common column is.

Example 1

The result:

Here, we performed the join based on the **common columns**. The database *can* do this as well and it is called a **natural join**.

Natural Join is an **Anti-pattern**

The natural join is **dangerously implicit**: the RDBMS picks the join columns, not you.

1. **You cannot see the join condition.** A reader has to know both schemas to know what the query matches on.
2. **Schema changes silently change the result.** Add a column with the same name to both tables (say `updated_at`) and the join matches on it too, so rows disappear. Rename the shared column and it becomes a `CROSS JOIN`.

Always write the join condition explicitly.

Example 2

We could repeat the same example, but we could instead specify the join condition. It is best to **qualify each column with its table**, as in $R.A$.

What about $R.A = S.A$?[4]

This join is called a theta join. It just means that we explicitly write out the join condition. Every join you write in this class must be a theta join.

[4] Note that in SQL we **must** qualify A with its table because A is ambiguous. We don't know which A we are considering.

Example 3

Now let's join on $R.B = S.C$ and reuse our Cartesian product from before.[5]

[5] In SQL though, we do not need to qualify these columns because B is unambiguously in R and C is unambiguously in S.

This Section

- Querying Multiple Relations with Joins
  - Cardinality of Relationships

- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Cardinality Also Matters for Predictability – 1:1

It is important to be able to estimate the number of rows in the output of a join for debugging.

We need to know the relationship between the tables and the matches.

uid	midterm
000000000	NULL
012345678	98
111111111	95
121212121	96
123456789	85
246802468	5
987654321	50
999999999	NULL

uid	lastname	firstname	mi	gpa	major	level	career
000000000	ROAD	RUNNER	NULL	NULL	NULL	UFR	UG
012345678	BRUIN	JOE	NULL	3.85	ECONOMICS	UJR	UG
111111111	BEAR	TED	E.	3.80	MATHEMATICS	GMT	GD
121212121	BLOCK	GENE	D.	4.00	BIOLOGY	CHN	CH
123456789	BRUIN	JOSIE	NULL	3.80	MATHEMATICS	GD2	GD
246802468	SCHMOE	JOE	NULL	2.00	SOCIOLOGY	UFR	UG
987654321	COYOTE	WILE	E.	1.90	SOCIOLOGY	UJR	UG
999999999	APPLESEED	JOHNNY	NULL	3.50	ENGLISH	USR	UG

This is a **one-to-one** mapping. For each UID there is one midterm.[6]

[6] This data is bruinbase normalized.

Cardinality Also Matters for Predictability – 1:M

department

dept_id	dept_name
1	Sales
2	Engineering
3	Marketing

employee

emp_id	emp_name	dept_id
101	Alice	2
102	Bob	2
103	Carol	1
104	Dave	2
105	Eve	3

This is a **one-to-many** mapping. For each department there may be multiple employees. Note that M:1 is very similar.

Cardinality Also Matters for Predictability – M:M

Students

student_id	student_name	course_id
1	Alice	101
1	Alice	102
2	Bob	101
2	Bob	103
3	Carol	102
3	Carol	103

Courses

course_id	course_name
101	Math
102	Physics
103	Chemistry

This is a **many-to-many** mapping. For each student there may be multiple courses, and for each course there may be multiple students.

Outline

- Querying Multiple Relations with Joins
  - Cardinality of Relationships

- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Joins in SQL

The general syntax for a `JOIN` in SQL is below:

```
SELECT
    ...
FROM table1
JOIN table2
ON join_condition
... Other JOIN ON statements if needed ...
... WHERE / GROUP BY / ORDER BY etc.
;
```

Of course, we can use `WHERE`, `GROUP BY` and friends.

Joins in SQL

Our order for writing the query is now complete:

```
SELECT
FROM
JOIN
ON
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
```

Joins in SQL

And the order of execution...

```
FROM
JOIN / ON (filtering during join)
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT
```

Table aliases can be defined in `FROM` and `JOIN` and can be used anywhere. Column aliases are still only available in `ORDER BY`.

Joins in SQL

There are many different kinds of joins. They can be classified on various criteria:[7]

Mismatch Handling		Join Condition	Set-based	Miscellaneous
INNER		Equi	Semi join	Self
OUTER	LEFT	Non-Equi	Anti join	CROSS
	RIGHT			NATURAL
	FULL			Theta

[7] There is no keyword for semi-join or anti-join in most RDBMS. They are supported natively with keyword in DuckDB, Impala and U-SQL. Most RDBMS run `IN/NOT IN` and `EXISTS/NOT EXISTS` as semi- and anti-joins.

This Section

- Querying Multiple Relations with Joins
  - Cardinality of Relationships

- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Antipattern: `CROSS JOIN`

There is nearly no reason to write a cross join. It is the database equivalent of the Cartesian product $R \times S$.

```
SELECT R.A, B,
       S.A, C, D
FROM R CROSS JOIN S;
```

```
SELECT R.A, B,
       S.A, C, D
FROM R JOIN S ON TRUE;
```

Some students use `CROSS JOIN` everywhere and then filter the results with `WHERE`. This does not receive any credit.

Antipattern: `CROSS JOIN`

It is much more common to end up in a smaller cross join because your join condition is not tight enough.

- This means you may end up with many, many, many more rows than you wanted. BAD!
- The situation may be so bad that you end up in the database equivalent of an infinite loop.
  - RAM / memory is exhausted
  - The operating system starts "swapping" and "thrashing" – the hard disk is used as memory. This is slow.
  - Eventually the server crashes.
- This situation is sometimes called a **Cartesian join**, **runaway join**, **join explosion** or **fanout**.

Recap

There is a lot going on. Let's recap:

- The purpose of joins
- The concept of a join key and how it differs from primary, unique and foreign keys.
- The concept of **natural join** and to **never use it**.
- The concept of a theta join where we **explicitly spell out the join condition**.
- The cardinality of relationships – 1:1, 1:M, M:M
- The general form of a SQL query for joins.
- Introducing several types of joins.

This Section

- Querying Multiple Relations with Joins
  - Cardinality of Relationships

- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Inner Equi Join

The inner equi join is the most common join you will write. And you know what? We just covered it in Examples 2 and 3!

The `INNER` keyword means that when joining tables R and S, **we only return rows where the values of the join key match in both tables.**

No match, no row.

Inner Equi Join

We can write the following query for Example 2.

```
SELECT R.A, B, C, D
FROM R
INNER JOIN S
ON R.A = S.A;
```

Inner Equi Join

- Since A exists in both R and S, we must qualify it with its table: `R.A`.
- If we want to join another table, we would add another `(INNER) JOIN...ON` clause
- This is an inner join because we do not include rows from R that do not have a match in S in the join key.
- This is an equi join because the join condition contains $=$ and only $=$.
- **Table aliases** can be created in `FROM` and `JOIN`. They can be used anywhere after. In some kinds of joins they **must** be used.
- Note that `WHERE`, `ON`, `GROUP BY` can use columns that are **not** in the `SELECT`.

Inner Equi Join

Suppose R and S instead contain the rows below. Then this query yields:

A	B
α	β
γ	η
ϵ	ζ

$\bowtie_{R.A=S.A}$

A	C	D
α	β	θ
α	η	κ
γ	ι	λ

$=$

$R.A$	B	$S.A$	C	D
α	β	α	β	θ
α	β	α	η	κ
γ	η	γ	ι	λ

- Notice that we get **two** rows where $R.A = S.A = \alpha$. This is because α appears twice in S.
- That is, **we do not stop once we find a match, we keep going until we reach the end of the table!**
- Since this is an equijoin, it is always that $R.A = S.A$, you can just `SELECT` one of them.

Inner Equi Join

We can also use column aliases[8] everywhere for our sanity. For Example 2:

```
SELECT
  R.A AS RA,
  R.B AS B,
  S.A AS SA,   -- dup since A is join key. Unnecessary
  S.C AS C,
  S.D AS D
FROM R
INNER JOIN S
ON R.A = S.A;
```

If you are brand new to SQL, I recommend using aliases everywhere.

---

[8] These are different from table aliases, such as `FROM R AS r`.

This Section

- Querying Multiple Relations with Joins
  - Cardinality of Relationships
- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Inner Non-Equi Join

The inner non-equi join is not as common, but is worth covering as the non-equijoin is used all over SQL. If **any** comparator in the `ON` is not an = it is a non-equijoin. Suppose we have these tables:

score

id	name	pct
101	Alice	94
102	Bob	81
103	Charlie	77
104	Diana	65
105	Eve	58
106	Frank	89.9

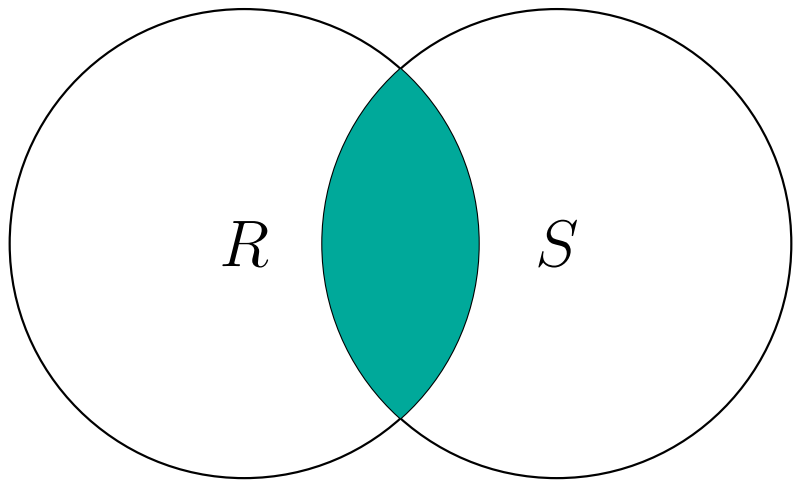
grade

min_pct	max_pct	grade
90	100	A
80	89	B
70	79	C
60	69	D
0	59	F

Demo: How would we compute each student's letter grade?

The INNER JOIN: Theory

Given two relations R and S, with join key k, the inner join concatenates records where the values of the join keys match.[9] In an equijoin, match means match on equality. For non-equijoins, match means "satisfies the join condition" on both relations.



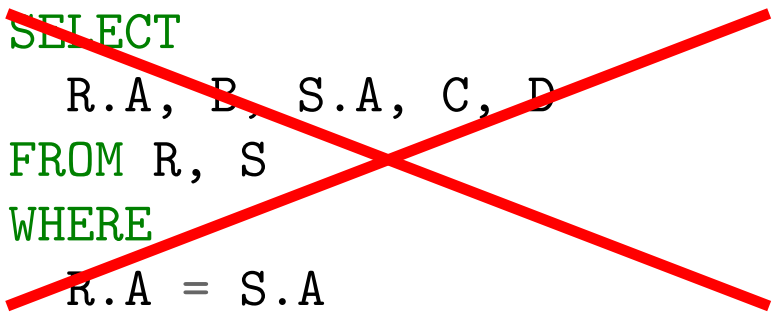
[9] Most joins we encounter are equijoins (all binary operators are $=$). By *match* I mean "values on both join keys that satisfy the join condition(s)."

This Section

- Querying Multiple Relations with Joins
  - Cardinality of Relationships
- Joins in SQL
  - Cross Join
  - Inner Equi Join
  - Inner Non-Equi Join
  - Antipatterns in Joins

Antipattern: Implicit / SQL-89 / Comma Join

You may see this form of a join instead. **Do not do this. We will not accept it. It is not good style and is old.**



```
SELECT
  R.A, B, S.A, C, D
FROM R, S
WHERE
  R.A = S.A
```

Don't be USING

The USING form allows joining when the join key has the same name on both sides of an inner equi join (like Example 2).

```
SELECT
  R.A,
  B,
  S.A,
  C,
  D
FROM R JOIN S
USING(A)
```

On exams you must explicitly state the join condition. Use JOIN...ON.

Set of Rules for Joins

Rules for joins. This is in addition to the rules we introduced last time:

- Explicitly specify the join condition, always.
- Never use `NATURAL JOIN`
- Never use the implicit (multiple tables in `FROM`) join.
- Never use `USING`.
- Use the correct join type, do not use shortcuts.

Before We Continue – Strategies for Writing Joins

Joins can make a query complex. This is how we should tackle any single-layer query.

1. Don't panic!
2. Remember that each join starts by forming all pairs of rows between two tables; and then
3. only keep rows that meet the join condition.

This will help you conceptualize what the result of the join should be. But there's more...

Before We Continue – Strategies for Writing Joins

The order of execution will also help you here in writing the query.

1. Start with `FROM`. You know which table the data comes from.
2. Next write the `JOIN`. This is the table we are joining with.
3. Then write the `ON` join condition. **This takes special care.**
4. Then write the `WHERE` filter condition to filter rows after the join.
5. Next write the `GROUP BY` if necessary, since you now have reduced data to work with.
6. Write the `HAVING`, if necessary, to filter on aggregates.
7. **Then** write the `SELECT` as it should be apparent what you want in the output with proper aliases.
8. Finally, write the `ORDER BY` if necessary
9. (Limit your results if needed)

Note that this process may be iterative if your query is not correct.

Another Approach

Write the SQL clause for each construct:

Construct	SQL Clause
`FROM`	
`JOIN`	
`ON`	
`WHERE`	
`GROUP BY`	
`HAVING`	
`SELECT`	

From Columns to Rows (`UNION`)

A join creates a table that is **wider** in that it may have more columns in the output.

We can also combine rows, to make the output **taller**, but this is not the same thing as a join.

From Columns to Rows (`UNION`)

Two ways:

1. `UNION` combines rows from two tables R and S **and removes duplicates**
2. `UNION ALL` does the same but **does not remove duplicates**.

Even if either table has a primary key, duplicates are permitted because the output is a **result set**, not a table.

Have a Fun and Safe Halloween!

